PEP 544 – Protocols: Structural subtyping (static duck typing)

Author: Ivan Levkivskyi <levkivskyi at gmail.com>, Jukka Lehtosalo <jukka.lehtosalo at iki.fi>, Łukasz Langa <lukasz at python.org>

BDFL-Delegate: Guido van Rossum <guido at python.org>

Discussions-To: Python-Dev list (https://mail.python.org/archives/list/python-dev@python.org/)

Status: Final

Type: Standards Track

Topic: Typing (../topic/typing/)

Created: 05-Mar-2017

Python-Version: 3.8

Resolution: Typing-SIG message (https://mail.python.org/archives/list/typing-sig@python.org/message/FDO4KFYWYQEP3U2HVVBEBR3SXPHQSHYR/)

Source: https://github.com/python/peps/blob/main/peps/pep-0544.rst

Last modified: 2025-02-01 07:28:42 UTC

Important ×

This PEP is a historical document: see Protocols (https://typing.python.org/en/latest/spec/protocol.html#protocols) and `typing.Protocol` (https://docs.python.org/3/library/typing.html#typing.Protocol) for up-to-date specs and documentation. Canonical typing specs are maintained at the typing specs site (https://typing.python.org/en/latest/spec/); runtime typing behaviour is described in the CPython documentation.

> See the typing specification update process (https://typing.python.org/en/latest/spec/meta.html) for how to propose changes to the typing spec.

Abstract (#abstract)

Type hints introduced in PEP 484 (../pep-0484/) can be used to specify type metadata for static type checkers and other third party tools. However, PEP 484 (../pep-0484/) only specifies the semantics of *nominal* subtyping. In this PEP we specify static and runtime semantics of protocol classes that will provide a support for *structural* subtyping (static duck typing).

Rationale and Goals (#rationale-and-goals)

Currently, PEP 484 (../pep-0484/) and the `typing` module [typing] (#typing) define abstract base classes for several common Python protocols such as `Iterable` and `Sized`. The problem with them is that a class has to be explicitly marked to support them, which is unpythonic and unlike what one would normally do in idiomatic dynamically typed Python code. For example, this conforms to PEP 484 (../pep-0484/):

```
from typing import Sized, Iterable, Iterator

class Bucket(Sized, Iterable[int]):
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...
```

The same problem appears with user-defined ABCs: they must be explicitly subclassed or registered. This is particularly difficult to do with library types as the type objects may be hidden deep in the implementation of the library. Also, extensive use of ABCs might impose additional runtime costs.

The intention of this PEP is to solve all these problems by allowing users to write the above code without explicit base classes in the class definition, allowing `Bucket` to be implicitly considered a subtype of both `Sized` and `Iterable[int]` by static type checkers using structural [wiki-structural] (#wiki-structural) subtyping:

```
from typing import Iterator, Iterable

class Bucket:
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...

def collect(items: Iterable[int]) -> int: ...
result: int = collect(Bucket())  # Passes type check
```

Note that ABCs in `typing` module already provide structural behavior at runtime, `isinstance(Bucket(), Iterable)` returns `True`. The main goal of this proposal is to support such behavior statically. The same functionality will be provided for user-defined protocols, as specified below. The above code with a protocol class matches common Python conventions much better. It is also automatically extensible and works with additional, unrelated classes that happen to implement the required protocol.

Nominal vs structural subtyping (#nominal-vs-structural-subtyping)

Structural subtyping is natural for Python programmers since it matches the runtime semantics of duck typing: an object that has certain properties is treated independently of its actual runtime class. However, as discussed in PEP 483 (../pep-0483/), both nominal and structural subtyping have their strengths and weaknesses. Therefore, in this PEP we *do not*

propose to replace the nominal subtyping described by PEP 484 (../pep-0484/) with structural subtyping completely. Instead, protocol classes as specified in this PEP complement normal classes, and users are free to choose where to apply a particular solution. See section on rejected (#pep-544-rejected) ideas at the end of this PEP for additional motivation.

Non-goals (#non-goals)

At runtime, protocol classes will be simple ABCs. There is no intent to provide sophisticated runtime instance and class checks against protocol classes. This would be difficult and error-prone and will contradict the logic of PEP 484 (../pep-0484/). As well, following PEP 484 (../pep-0484/) and PEP 526 (../pep-0526/) we state that protocols are **completely optional**:

- No runtime semantics will be imposed for variables or parameters annotated with a protocol class.
- Any checks will be performed only by third-party type checkers and other tools.
- Programmers are free to not use them even if they use type annotations.
- There is no intent to make protocols non-optional in the future.

To reiterate, providing complex runtime semantics for protocol classes is not a goal of this PEP, the main goal is to provide a support and standards for *static* structural subtyping. The possibility to use protocols in the runtime context as ABCs is rather a minor bonus that exists mostly to provide a seamless transition for projects that already use ABCs.

Existing Approaches to Structural Subtyping (#existing-approaches-to-structural-subtyping)

Before describing the actual specification, we review and comment on existing approaches related to structural subtyping in Python and other languages:

- `zope.interface` [zope-interfaces] (#zope-interfaces) was one of the first widely used approaches to structural subtyping in Python. It is implemented by providing special classes to distinguish interface classes from normal classes, to

mark interface attributes, and to explicitly declare implementation. For example:

```
from zope.interface import Interface, Attribute, implementer

class IEmployee(Interface):

    name = Attribute("Name of employee")

    def do(work):
        """Do some work"""

@implementer(IEmployee)
class Employee:

    name = 'Anonymous'

    def do(self, work):
        return work.start()
```

Zope interfaces support various contracts and constraints for interface classes. For example:

```
from zope.interface import invariant

def required_contact(obj):
    if not (obj.email or obj.phone):
        raise Exception("At least one contact info is required")

class IPerson(Interface):

    name = Attribute("Name")
    email = Attribute("Email Address")
    phone = Attribute("Phone Number")

    invariant(required_contact)
```

Even more detailed invariants are supported. However, Zope interfaces rely entirely on runtime validation. Such focus on runtime properties goes beyond the scope of the current proposal, and static support for invariants might be difficult to implement. However, the idea of marking an interface class with a special base class is reasonable and easy to implement both statically and at runtime.

- Python abstract base classes [abstract-classes] (#abstract-classes) are the standard library tool to provide some functionality similar to structural subtyping. The drawback of this approach is the necessity to either subclass the abstract class or register an implementation explicitly:

```
from abc import ABC

class MyTuple(ABC):
    pass

MyTuple.register(tuple)

assert issubclass(tuple, MyTuple)
assert isinstance((), MyTuple)
```

  As mentioned in the rationale (#pep-544-rationale), we want to avoid such necessity, especially in static context. However, in a runtime context, ABCs are good candidates for protocol classes and they are already used extensively in the `typing` module.

- Abstract classes defined in `collections.abc` module [collections-abc] (#collections-abc) are slightly more advanced since they implement a custom `__subclasshook__()` method that allows runtime structural checks without explicit registration:

```
from collections.abc import Iterable

class MyIterable:
    def __iter__(self):
        return []

assert isinstance(MyIterable(), Iterable)
```

Such behavior seems to be a perfect fit for both runtime and static behavior of protocols. As discussed in rationale (#pep-544-rationale), we propose to add static support for such behavior. In addition, to allow users to achieve such runtime behavior for *user-defined* protocols a special `@runtime_checkable` decorator will be provided, see detailed discussion (#discussion) below.

- TypeScript [typescript] (#typescript) provides support for user-defined classes and interfaces. Explicit implementation declaration is not required and structural subtyping is verified statically. For example:

```
interface LabeledItem {
    label: string;
    size?: int;
}

function printLabel(obj: LabeledItem) {
    console.log(obj.label);
}

let myObj = {size: 10, label: "Size 10 Object"};
printLabel(myObj);
```

Note that optional interface members are supported. Also, TypeScript prohibits redundant members in implementations. While the idea of optional members looks interesting, it would complicate this proposal and it is not clear how useful it will be. Therefore, it is proposed to postpone this; see rejected (#pep-544-rejected) ideas. In general, the idea of static protocol checking without runtime implications looks reasonable, and basically this proposal follows the same line.

- Go [golang] (#golang) uses a more radical approach and makes interfaces the primary way to provide type information. Also, assignments are used to explicitly ensure implementation:

```
type SomeInterface interface {
    SomeMethod() ([]byte, error)
}

if _, ok := someval.(SomeInterface); ok {
    fmt.Printf("value implements some interface")
}
```

Both these ideas are questionable in the context of this proposal. See the section on rejected (#pep-544-rejected) ideas.

Specification (#specification)

Terminology (#terminology)

We propose to use the term *protocols* for types supporting structural subtyping. The reason is that the term *iterator protocol*, for example, is widely understood in the community, and coming up with a new term for this concept in a statically typed context would just create confusion.

This has the drawback that the term *protocol* becomes overloaded with two subtly different meanings: the first is the traditional, well-known but slightly fuzzy concept of protocols such as iterator; the second is the more explicitly defined concept of protocols in statically typed code. The distinction is not important most of the time, and in other cases we propose to just add a qualifier such as *protocol classes* when referring to the static type concept.

If a class includes a protocol in its MRO, the class is called an *explicit* subclass of the protocol. If a class is a structural subtype of a protocol, it is said to implement the protocol and to be compatible with a protocol. If a class is compatible with a protocol but the protocol is not included in the MRO, the class is an *implicit* subtype of the protocol. (Note that one can explicitly subclass a protocol and still not implement it if a protocol attribute is set to `None` in the subclass, see Python [data-model] (#data-model) for details.)

The attributes (variables and methods) of a protocol that are mandatory for other class in order to be considered a structural subtype are called protocol members.

Defining a protocol (#defining-a-protocol)

Protocols are defined by including a special new class `typing.Protocol` (an instance of `abc.ABCMeta`) in the base classes list, typically at the end of the list. Here is a simple example:

```
from typing import Protocol

class SupportsClose(Protocol):
    def close(self) -> None:
        ...
```

Now if one defines a class `Resource` with a `close()` method that has a compatible signature, it would implicitly be a subtype of `SupportsClose`, since the structural subtyping is used for protocol types:

```
class Resource:
    ...
    def close(self) -> None:
        self.file.close()
        self.lock.release()
```

Apart from few restrictions explicitly mentioned below, protocol types can be used in every context where a normal types can:

```
def close_all(things: Iterable[SupportsClose]) -> None:
    for t in things:
        t.close()

f = open('foo.txt')
r = Resource()
close_all([f, r])  # OK!
close_all([1])     # Error: 'int' has no 'close' method
```

Note that both the user-defined class `Resource` and the built-in `IO` type (the return type of `open()`) are considered subtypes of `SupportsClose`, because they provide a `close()` method with a compatible type signature.

Protocol members (#protocol-members)

All methods defined in the protocol class body are protocol members, both normal and decorated with `@abstractmethod`. If any parameters of a protocol method are not annotated, then their types are assumed to be `Any` (see PEP 484 (../pep-0484/)). Bodies of protocol methods are type checked. An abstract method that should not be called via `super()` ought to raise `NotImplementedError`. Example:

```
from typing import Protocol
from abc import abstractmethod

class Example(Protocol):
    def first(self) -> int:     # This is a protocol member
        return 42

    @abstractmethod
    def second(self) -> int:    # Method without a default implementation
        raise NotImplementedError
```

Static methods, class methods, and properties are equally allowed in protocols.

To define a protocol variable, one can use PEP 526 (../pep-0526/) variable annotations in the class body. Additional attributes *only* defined in the body of a method by assignment via `self` are not allowed. The rationale for this is that the protocol class implementation is often not shared by subtypes, so the interface should not depend on the default implementation. Examples:

```
from typing import Protocol, List

class Template(Protocol):
    name: str        # This is a protocol member
    value: int = 0   # This one too (with default)

    def method(self) -> None:
        self.temp: List[int] = [] # Error in type checker

class Concrete:
    def __init__(self, name: str, value: int) -> None:
        self.name = name
        self.value = value

    def method(self) -> None:
        return

var: Template = Concrete('value', 42)  # OK
```

To distinguish between protocol class variables and protocol instance variables, the special `ClassVar` annotation should be used as specified by PEP 526 (../pep-0526/). By default, protocol variables as defined above are considered readable and writable. To define a read-only protocol variable, one can use an (abstract) property.

Explicitly declaring implementation (#explicitly-declaring-implementation)

To explicitly declare that a certain class implements a given protocol, it can be used as a regular base class. In this case a class could use default implementations of protocol members. Static analysis tools are expected to automatically detect that a class implements a given protocol. So while it's possible to subclass a protocol explicitly, it's *not necessary* to do so for the sake of type-checking.

The default implementations cannot be used if the subtype relationship is implicit and only via structural subtyping – the semantics of inheritance is not changed. Examples:

```
class PColor(Protocol):
    @abstractmethod
    def draw(self) -> str:
        ...
    def complex_method(self) -> int:
        # some complex code here
        ...

class NiceColor(PColor):
    def draw(self) -> str:
        return "deep blue"

class BadColor(PColor):
    def draw(self) -> str:
        return super().draw()  # Error, no default implementation

class ImplicitColor:   # Note no 'PColor' base here
    def draw(self) -> str:
        return "probably gray"
    def complex_method(self) -> int:
        # class needs to implement this
        ...

nice: NiceColor
another: ImplicitColor

def represent(c: PColor) -> None:
    print(c.draw(), c.complex_method())

represent(nice) # OK
represent(another) # Also OK
```

Note that there is little difference between explicit and implicit subtypes, the main benefit of explicit subclassing is to get some protocol methods "for free". In addition, type checkers can statically verify that the class actually implements the protocol correctly:

```
class RGB(Protocol):
    rgb: Tuple[int, int, int]

    @abstractmethod
    def intensity(self) -> int:
        return 0

class Point(RGB):
    def __init__(self, red: int, green: int, blue: str) -> None:
        self.rgb = red, green, blue  # Error, 'blue' must be 'int'

    # Type checker might warn that 'intensity' is not defined
```

A class can explicitly inherit from multiple protocols and also from normal classes. In this case methods are resolved using normal MRO and a type checker verifies that all subtyping are correct. The semantics of `@abstractmethod` is not changed, all of them must be implemented by an explicit subclass before it can be instantiated.

Merging and extending protocols (#merging-and-extending-protocols)

The general philosophy is that protocols are mostly like regular ABCs, but a static type checker will handle them specially. Subclassing a protocol class would not turn the subclass into a protocol unless it also has `typing.Protocol` as an explicit base class. Without this base, the class is "downgraded" to a regular ABC that cannot be used with structural subtyping. The rationale for this rule is that we don't want to accidentally have some class act as a protocol just because one of its base classes happens to be one. We still slightly prefer nominal subtyping over structural subtyping in the static typing world.

A subprotocol can be defined by having *both* one or more protocols as immediate base classes and also having `typing.Protocol` as an immediate base class:

```
from typing import Sized, Protocol

class SizedAndClosable(Sized, Protocol):
    def close(self) -> None:
        ...
```

Now the protocol `SizedAndClosable` is a protocol with two methods, `__len__` and `close`. If one omits `Protocol` in the base class list, this would be a regular (non-protocol) class that must implement `Sized`. Alternatively, one can implement `SizedAndClosable` protocol by merging the `SupportsClose` protocol from the example in the definition (#definition) section with `typing.Sized`:

```
from typing import Sized

class SupportsClose(Protocol):
    def close(self) -> None:
        ...

class SizedAndClosable(Sized, SupportsClose, Protocol):
    pass
```

The two definitions of `SizedAndClosable` are equivalent. Subclass relationships between protocols are not meaningful when considering subtyping, since structural compatibility is the criterion, not the MRO.

If `Protocol` is included in the base class list, all the other base classes must be protocols. A protocol can't extend a regular class, see rejected (#pep-544-rejected) ideas for rationale. Note that rules around explicit subclassing are different from regular ABCs, where abstractness is simply defined by having at least one abstract method being unimplemented.

Protocol classes must be marked *explicitly*.

Generic protocols (#generic-protocols)

Generic protocols are important. For example, `SupportsAbs`, `Iterable` and `Iterator` are generic protocols. They are defined similar to normal non-protocol generic types:

```
class Iterable(Protocol[T]):
    @abstractmethod
    def __iter__(self) -> Iterator[T]:
        ...
```

`Protocol[T, S, ...]` is allowed as a shorthand for `Protocol, Generic[T, S, ...]`.

User-defined generic protocols support explicitly declared variance. Type checkers will warn if the inferred variance is different from the declared variance. Examples:

```
T = TypeVar('T')
T_co = TypeVar('T_co', covariant=True)
T_contra = TypeVar('T_contra', contravariant=True)

class Box(Protocol[T_co]):
    def content(self) -> T_co:
        ...

box: Box[float]
second_box: Box[int]
box = second_box  # This is OK due to the covariance of 'Box'.

class Sender(Protocol[T_contra]):
    def send(self, data: T_contra) -> int:
        ...

sender: Sender[float]
new_sender: Sender[int]
new_sender = sender  # OK, 'Sender' is contravariant.

class Proto(Protocol[T]):
    attr: T  # this class is invariant, since it has a mutable attribute

var: Proto[float]
another_var: Proto[int]
var = another_var  # Error! 'Proto[float]' is incompatible with 'Proto[int]'.
```

Note that unlike nominal classes, de facto covariant protocols cannot be declared as invariant, since this can break transitivity of subtyping (see rejected (#pep-544-rejected) ideas for details). For example:

```
T = TypeVar('T')

class AnotherBox(Protocol[T]):  # Error, this protocol is covariant in T,
    def content(self) -> T:     # not invariant.
        ...
```

Recursive protocols (#recursive-protocols)

Recursive protocols are also supported. Forward references to the protocol class names can be given as strings as specified by PEP 484 (../pep-0484/). Recursive protocols are useful for representing self-referential data structures like trees in an abstract fashion:

```
class Traversable(Protocol):
    def leaves(self) -> Iterable['Traversable']:
        ...
```

Note that for recursive protocols, a class is considered a subtype of the protocol in situations where the decision depends on itself. Continuing the previous example:

```
class SimpleTree:
    def leaves(self) -> List['SimpleTree']:
        ...

root: Traversable = SimpleTree()  # OK

class Tree(Generic[T]):
    def leaves(self) -> List['Tree[T]']:
        ...

def walk(graph: Traversable) -> None:
    ...
tree: Tree[float] = Tree()
walk(tree)  # OK, 'Tree[float]' is a subtype of 'Traversable'
```

Self-types in protocols (#self-types-in-protocols)

The self-types in protocols follow the corresponding specification (../pep-0484/#annotating-instance-and-class-methods) of PEP 484 (../pep-0484/). For example:

```
C = TypeVar('C', bound='Copyable')
class Copyable(Protocol):
    def copy(self: C) -> C:

class One:
    def copy(self) -> 'One':
        ...

T = TypeVar('T', bound='Other')
class Other:
    def copy(self: T) -> T:
        ...

c: Copyable
c = One()  # OK
c = Other()  # Also OK
```

Callback protocols (#callback-protocols)

Protocols can be used to define flexible callback types that are hard (or even impossible) to express using the `Callable[...]` syntax specified by PEP 484 (../pep-0484/), such as variadic, overloaded, and complex generic callbacks. They can be defined as protocols with a `__call__` member:

```
from typing import Optional, List, Protocol

class Combiner(Protocol):
    def __call__(self, *vals: bytes,
                 maxlen: Optional[int] = None) -> List[bytes]: ...

def good_cb(*vals: bytes, maxlen: Optional[int] = None) -> List[bytes]:
    ...
def bad_cb(*vals: bytes, maxitems: Optional[int]) -> List[bytes]:
    ...

comb: Combiner = good_cb  # OK
comb = bad_cb  # Error! Argument 2 has incompatible type because of
               # different name and kind in the callback
```

Callback protocols and `Callable[...]` types can be used interchangeably.

Using Protocols (#using-protocols)

Subtyping relationships with other types (#subtyping-relationships-with-other-types)

Protocols cannot be instantiated, so there are no values whose runtime type is a protocol. For variables and parameters with protocol types, subtyping relationships are subject to the following rules:

- A protocol is never a subtype of a concrete type.
- A concrete type `X` is a subtype of protocol `P` if and only if `X` implements all protocol members of `P` with compatible types. In other words, subtyping with respect to a protocol is always structural.
- A protocol `P1` is a subtype of another protocol `P2` if `P1` defines all protocol members of `P2` with compatible types.

Generic protocol types follow the same rules of variance as non-protocol types. Protocol types can be used in all contexts where any other types can be used, such as in `Union`, `ClassVar`, type variables bounds, etc. Generic protocols follow the rules for generic abstract classes, except for using structural compatibility instead of compatibility defined by inheritance relationships.

Static type checkers will recognize protocol implementations, even if the corresponding protocols are *not imported*:

```
# file lib.py
from typing import Sized

T = TypeVar('T', contravariant=True)
class ListLike(Sized, Protocol[T]):
    def append(self, x: T) -> None:
        pass

def populate(lst: ListLike[int]) -> None:
    ...

# file main.py
from lib import populate  # Note that ListLike is NOT imported

class MockStack:
    def __len__(self) -> int:
        return 42
    def append(self, x: int) -> None:
        print(x)

populate([1, 2, 3])    # Passes type check
populate(MockStack())  # Also OK
```

Unions and intersections of protocols (#unions-and-intersections-of-protocols)

`Union` of protocol classes behaves the same way as for non-protocol classes. For example:

```
from typing import Union, Optional, Protocol

class Exitable(Protocol):
    def exit(self) -> int:
        ...
class Quittable(Protocol):
    def quit(self) -> Optional[int]:
        ...

def finish(task: Union[Exitable, Quittable]) -> int:
    ...
class DefaultJob:
    ...
    def quit(self) -> int:
        return 0
finish(DefaultJob()) # OK
```

One can use multiple inheritance to define an intersection of protocols. Example:

```
from typing import Iterable, Hashable

class HashableFloats(Iterable[float], Hashable, Protocol):
    pass

def cached_func(args: HashableFloats) -> float:
    ...
cached_func((1, 2, 3)) # OK, tuple is both hashable and iterable
```

If this will prove to be a widely used scenario, then a special intersection type construct could be added in future as specified by PEP 483 (../pep-0483/), see rejected (#pep-544-rejected) ideas for more details.

`Type[]` and class objects vs protocols (#type-and-class-objects-vs-protocols)

Variables and parameters annotated with `Type[Proto]` accept only concrete (non-protocol) subtypes of `Proto`. The main reason for this is to allow instantiation of parameters with such type. For example:

```
class Proto(Protocol):
    @abstractmethod
    def meth(self) -> int:
        ...
class Concrete:
    def meth(self) -> int:
        return 42

def fun(cls: Type[Proto]) -> int:
    return cls().meth() # OK
fun(Proto)              # Error
fun(Concrete)           # OK
```

The same rule applies to variables:

```
var: Type[Proto]
var = Proto    # Error
var = Concrete # OK
var().meth()   # OK
```

Assigning an ABC or a protocol class to a variable is allowed if it is not explicitly typed, and such assignment creates a type alias. For normal (non-abstract) classes, the behavior of `Type[]` is not changed.

A class object is considered an implementation of a protocol if accessing all members on it results in types compatible with the protocol members. For example:

```
from typing import Any, Protocol

class ProtoA(Protocol):
    def meth(self, x: int) -> int: ...
class ProtoB(Protocol):
    def meth(self, obj: Any, x: int) -> int: ...

class C:
    def meth(self, x: int) -> int: ...

a: ProtoA = C  # Type check error, signatures don't match!
b: ProtoB = C  # OK
```

`NewType()` and type aliases (#newtype-and-type-aliases)

Protocols are essentially anonymous. To emphasize this point, static type checkers might refuse protocol classes inside `NewType()` to avoid an illusion that a distinct type is provided:

```
from typing import NewType, Protocol, Iterator

class Id(Protocol):
    code: int
    secrets: Iterator[bytes]

UserId = NewType('UserId', Id)  # Error, can't provide distinct type
```

In contrast, type aliases are fully supported, including generic type aliases:

```
from typing import TypeVar, Reversible, Iterable, Sized

T = TypeVar('T')
class SizedIterable(Iterable[T], Sized, Protocol):
    pass
CompatReversible = Union[Reversible[T], SizedIterable[T]]
```

Modules as implementations of protocols (#modules-as-implementations-of-protocols)

A module object is accepted where a protocol is expected if the public interface of the given module is compatible with the expected protocol. For example:

```
# file default_config.py
timeout = 100
one_flag = True
other_flag = False

# file main.py
import default_config
from typing import Protocol

class Options(Protocol):
    timeout: int
    one_flag: bool
    other_flag: bool

def setup(options: Options) -> None:
    ...

setup(default_config)  # OK
```

To determine compatibility of module level functions, the `self` argument of the corresponding protocol methods is dropped. For example:

```
# callbacks.py
def on_error(x: int) -> None:
    ...
def on_success() -> None:
    ...

# main.py
import callbacks
from typing import Protocol

class Reporter(Protocol):
    def on_error(self, x: int) -> None:
        ...
    def on_success(self) -> None:
        ...

rp: Reporter = callbacks  # Passes type check
```

`@runtime_checkable` decorator and narrowing types by `isinstance()` (#runtime-checkable-decorator-and-narrowing-types-by-isinstance)

The default semantics is that `isinstance()` and `issubclass()` fail for protocol types. This is in the spirit of duck typing – protocols basically would be used to model duck typing statically, not explicitly at runtime.

However, it should be possible for protocol types to implement custom instance and class checks when this makes sense, similar to how `Iterable` and other ABCs in `collections.abc` and `typing` already do it, but this is limited to non-generic and unsubscripted generic protocols (`Iterable` is statically equivalent to `Iterable[Any]`). The `typing` module will

define a special `@runtime_checkable` class decorator that provides the same semantics for class and instance checks as for `collections.abc` classes, essentially making them "runtime protocols":

```
from typing import runtime_checkable, Protocol

@runtime_checkable
class SupportsClose(Protocol):
    def close(self):
        ...

assert isinstance(open('some/file'), SupportsClose)
```

Note that instance checks are not 100% reliable statically, this is why this behavior is opt-in, see section on rejected (#pep-544-rejected) ideas for examples. The most type checkers can do is to treat `isinstance(obj, Iterator)` roughly as a simpler way to write `hasattr(x, '__iter__') and hasattr(x, '__next__')`. To minimize the risks for this feature, the following rules are applied.

Definitions:

- *Data, and non-data protocols*: A protocol is called non-data protocol if it only contains methods as members (for example `Sized`, `Iterator`, etc). A protocol that contains at least one non-method member (like `x: int`) is called a data protocol.
- *Unsafe overlap*: A type `X` is called unsafely overlapping with a protocol `P`, if `X` is not a subtype of `P`, but it is a subtype of the type erased version of `P` where all members have type `Any`. In addition, if at least one element of a union unsafely overlaps with a protocol `P`, then the whole union is unsafely overlapping with `P`.

Specification:

- A protocol can be used as a second argument in `isinstance()` and `issubclass()` only if it is explicitly opt-in by `@runtime_checkable` decorator. This requirement exists because protocol checks are not type safe in case of dynamically set attributes, and because type checkers can only prove that an `isinstance()` check is safe only for a given class, not for all its subclasses.
- `isinstance()` can be used with both data and non-data protocols, while `issubclass()` can be used only with non-data protocols. This restriction exists because some data attributes can be set on an instance in constructor and this information is not always available on the class object.
- Type checkers should reject an `isinstance()` or `issubclass()` call, if there is an unsafe overlap between the type of the first argument and the protocol.
- Type checkers should be able to select a correct element from a union after a safe `isinstance()` or `issubclass()` call. For narrowing from non-union types, type checkers can use their best judgement (this is intentionally unspecified, since a precise specification would require intersection types).

Using Protocols in Python 2.7 - 3.5 (#using-protocols-in-python-2-7-3-5)

Variable annotation syntax was added in Python 3.6, so that the syntax for defining protocol variables proposed in specification (#specification) section can't be used if support for earlier versions is needed. To define these in a manner compatible with older versions of Python one can use properties. Properties can be settable and/or abstract if needed:

```
class Foo(Protocol):
    @property
    def c(self) -> int:
        return 42         # Default value can be provided for property...

    @abstractproperty
    def d(self) -> int:   # ... or it can be abstract
        return 0
```

Also function type comments can be used as per PEP 484 (../pep-0484/) (for example to provide compatibility with Python 2). The `typing` module changes proposed in this PEP will also be backported to earlier versions via the backport currently available on PyPI.

Runtime Implementation of Protocol Classes (#runtime-implementation-of-protocol-classes)

Implementation details (#implementation-details)

The runtime implementation could be done in pure Python without any effects on the core interpreter and standard library except in the `typing` module, and a minor update to `collections.abc`:

- Define class `typing.Protocol` similar to `typing.Generic`.
- Implement functionality to detect whether a class is a protocol or not. Add a class attribute `_is_protocol = True` if that is the case. Verify that a protocol class only has protocol base classes in the MRO (except for object).
- Implement `@runtime_checkable` that allows `__subclasshook__()` performing structural instance and subclass checks as in `collections.abc` classes.
- All structural subtyping checks will be performed by static type checkers, such as `mypy` [mypy] (#mypy). No additional support for protocol validation will be provided at runtime.

Changes in the typing module (#changes-in-the-typing-module)

The following classes in `typing` module will be protocols:

- `Callable`
- `Awaitable`
- `Iterable, Iterator`

- `AsyncIterable, AsyncIterator`
- `Hashable`
- `Sized`
- `Container`
- `Collection`
- `Reversible`
- `ContextManager, AsyncContextManager`
- `SupportsAbs` (and other `Supports*` classes)

Most of these classes are small and conceptually simple. It is easy to see what are the methods these protocols implement, and immediately recognize the corresponding runtime protocol counterpart. Practically, few changes will be needed in `typing` since some of these classes already behave the necessary way at runtime. Most of these will need to be updated only in the corresponding `typeshed` stubs [typeshed] (#typeshed).

All other concrete generic classes such as `List, Set, IO, Deque`, etc are sufficiently complex that it makes sense to keep them non-protocols (i.e. require code to be explicit about them). Also, it is too easy to leave some methods unimplemented by accident, and explicitly marking the subclass relationship allows type checkers to pinpoint the missing implementations.

Introspection (#introspection)

The existing class introspection machinery (`dir, __annotations__` etc) can be used with protocols. In addition, all introspection tools implemented in the `typing` module will support protocols. Since all attributes need to be defined in the class body based on this proposal, protocol classes will have even better perspective for introspection than regular

classes where attributes can be defined implicitly – protocol attributes can't be initialized in ways that are not visible to introspection (using `setattr()`, assignment via `self`, etc.). Still, some things like types of attributes will not be visible at runtime in Python 3.5 and earlier, but this looks like a reasonable limitation.

There will be only limited support of `isinstance()` and `issubclass()` as discussed above (these will *always* fail with `TypeError` for subscripted generic protocols, since a reliable answer could not be given at runtime in this case). But together with other introspection tools this give a reasonable perspective for runtime type checking tools.

Rejected/Postponed Ideas (#rejected-postponed-ideas)

The ideas in this section were previously discussed in [several] (#several) [discussions] (#discussions) [elsewhere] (#elsewhere).

Make every class a protocol by default (#make-every-class-a-protocol-by-default)

Some languages such as Go make structural subtyping the only or the primary form of subtyping. We could achieve a similar result by making all classes protocols by default (or even always). However we believe that it is better to require classes to be explicitly marked as protocols, for the following reasons:

- Protocols don't have some properties of regular classes. In particular, `isinstance()`, as defined for normal classes, is based on the nominal hierarchy. In order to make everything a protocol by default, and have `isinstance()` work would require changing its semantics, which won't happen.
- Protocol classes should generally not have many method implementations, as they describe an interface, not an implementation. Most classes have many method implementations, making them bad protocol classes.
- Experience suggests that many classes are not practical as protocols anyway, mainly because their interfaces are too large, complex or implementation-oriented (for example, they may include de facto private attributes and

methods without a `__` prefix).

- Most actually useful protocols in existing Python code seem to be implicit. The ABCs in `typing` and `collections.abc` are rather an exception, but even they are recent additions to Python and most programmers do not use them yet.
- Many built-in functions only accept concrete instances of `int` (and subclass instances), and similarly for other built-in classes. Making `int` a structural type wouldn't be safe without major changes to the Python runtime, which won't happen.

Protocols subclassing normal classes (#protocols-subclassing-normal-classes)

The main rationale to prohibit this is to preserve transitivity of subtyping, consider this example:

```
from typing import Protocol

class Base:
    attr: str

class Proto(Base, Protocol):
    def meth(self) -> int:
        ...

class C:
    attr: str
    def meth(self) -> int:
        return 0
```

Now, `C` is a subtype of `Proto`, and `Proto` is a subtype of `Base`. But `C` cannot be a subtype of `Base` (since the latter is not a protocol). This situation would be really weird. In addition, there is an ambiguity about whether attributes of `Base` should become protocol members of `Proto`.

Support optional protocol members (#support-optional-protocol-members)

We can come up with examples where it would be handy to be able to say that a method or data attribute does not need to be present in a class implementing a protocol, but if it is present, it must conform to a specific signature or type. One could use a `hasattr()` check to determine whether they can use the attribute on a particular instance.

Languages such as TypeScript have similar features and apparently they are pretty commonly used. The current realistic potential use cases for protocols in Python don't require these. In the interest of simplicity, we propose to not support optional methods or attributes. We can always revisit this later if there is an actual need.

Allow only protocol methods and force use of getters and setters (#allow-only-protocol-methods-and-force-use-of-getters-and-setters)

One could argue that protocols typically only define methods, but not variables. However, using getters and setters in cases where only a simple variable is needed would be quite unpythonic. Moreover, the widespread use of properties (that often act as type validators) in large code bases is partially due to previous absence of static type checkers for Python, the problem that PEP 484 (../pep-0484/) and this PEP are aiming to solve. For example:

```
# without static types

class MyClass:
    @property
    def my_attr(self):
        return self._my_attr
    @my_attr.setter
    def my_attr(self, value):
        if not isinstance(value, int):
            raise ValidationError("An integer expected for my_attr")
        self._my_attr = value

# with static types

class MyClass:
    my_attr: int
```

Support non-protocol members (#support-non-protocol-members)

There was an idea to make some methods "non-protocol" (i.e. not necessary to implement, and inherited in explicit subclassing), but it was rejected, since this complicates things. For example, consider this situation:

```
class Proto(Protocol):
    @abstractmethod
    def first(self) -> int:
        raise NotImplementedError
    def second(self) -> int:
        return self.first() + 1

def fun(arg: Proto) -> None:
    arg.second()
```

The question is should this be an error? We think most people would expect this to be valid. Therefore, to be on the safe side, we need to require both methods to be implemented in implicit subclasses. In addition, if one looks at definitions in `collections.abc`, there are very few methods that could be considered "non-protocol". Therefore, it was decided to not introduce "non-protocol" methods.

There is only one downside to this: it will require some boilerplate for implicit subtypes of "large" protocols. But, this doesn't apply to "built-in" protocols that are all "small" (i.e. have only few abstract methods). Also, such style is discouraged for user-defined protocols. It is recommended to create compact protocols and combine them.

Make protocols interoperable with other approaches (#make-protocols-interoperable-with-other-approaches)

The protocols as described here are basically a minimal extension to the existing concept of ABCs. We argue that this is the way they should be understood, instead of as something that *replaces* Zope interfaces, for example. Attempting such interoperabilities will significantly complicate both the concept and the implementation.

On the other hand, Zope interfaces are conceptually a superset of protocols defined here, but using an incompatible syntax to define them, because before PEP 526 (../pep-0526/) there was no straightforward way to annotate attributes. In the 3.6+ world, `zope.interface` might potentially adopt the `Protocol` syntax. In this case, type checkers could be taught to recognize interfaces as protocols and make simple structural checks with respect to them.

Use assignments to check explicitly that a class implements a protocol (#use-assignments-to-check-explicitly-that-a-class-implements-a-protocol)

In the Go language the explicit checks for implementation are performed via dummy assignments [golang] (#golang). Such a way is also possible with the current proposal. Example:

```
class A:
    def __len__(self) -> float:
        return ...

_: Sized = A()  # Error: A.__len__ doesn't conform to 'Sized'
                # (Incompatible return type 'float')
```

This approach moves the check away from the class definition and it almost requires a comment as otherwise the code probably would not make any sense to an average reader – it looks like dead code. Besides, in the simplest form it requires one to construct an instance of `A`, which could be problematic if this requires accessing or allocating some resources such as files or sockets. We could work around the latter by using a cast, for example, but then the code would be ugly. Therefore, we discourage the use of this pattern.

Support `isinstance()` checks by default (#support-isinstance-checks-by-default)

The problem with this is instance checks could be unreliable, except for situations where there is a common signature convention such as `Iterable`. For example:

```
class P(Protocol):
    def common_method_name(self, x: int) -> int: ...

class X:
    <a bunch of methods>
    def common_method_name(self) -> None: ... # Note different signature

def do_stuff(o: Union[P, X]) -> int:
    if isinstance(o, P):
        return o.common_method_name(1)  # Results in TypeError not caught
                                        # statically if o is an X instance.
```

Another potentially problematic case is assignment of attributes *after* instantiation:

```
class P(Protocol):
    x: int

class C:
    def initialize(self) -> None:
        self.x = 0

c = C()
isinstance(c, P)  # False
c.initialize()
isinstance(c, P)  # True

def f(x: Union[P, int]) -> None:
    if isinstance(x, P):
        # Static type of x is P here.
        ...
    else:
        # Static type of x is int, but can be other type at runtime...
        print(x + 1)

f(C())  # ...causing a TypeError.
```

We argue that requiring an explicit class decorator would be better, since one can then attach warnings about problems like this in the documentation. The user would be able to evaluate whether the benefits outweigh the potential for confusion for each protocol and explicitly opt in – but the default behavior would be safer. Finally, it will be easy to make this behavior default if necessary, while it might be problematic to make it opt-in after being default.

Provide a special intersection type construct (#provide-a-special-intersection-type-construct)

There was an idea to allow `Proto = All[Proto1, Proto2, ...]` as a shorthand for:

```
class Proto(Proto1, Proto2, ..., Protocol):
    pass
```

However, it is not yet clear how popular/useful it will be and implementing this in type checkers for non-protocol classes could be difficult. Finally, it will be very easy to add this later if needed.

Prohibit explicit subclassing of protocols by non-protocols (#prohibit-explicit-subclassing-of-protocols-by-non-protocols)

This was rejected for the following reasons:

- Backward compatibility: People are already using ABCs, including generic ABCs from `typing` module. If we prohibit explicit subclassing of these ABCs, then quite a lot of code will break.
- Convenience: There are existing protocol-like ABCs (that may be turned into protocols) that have many useful "mix-in" (non-abstract) methods. For example, in the case of `Sequence` one only needs to implement `__getitem__` and `__len__` in an explicit subclass, and one gets `__iter__`, `__contains__`, `__reversed__`, `index`, and `count` for free.
- Explicit subclassing makes it explicit that a class implements a particular protocol, making subtyping relationships easier to see.
- Type checkers can warn about missing protocol members or members with incompatible types more easily, without having to use hacks like dummy assignments discussed above in this section.
- Explicit subclassing makes it possible to force a class to be considered a subtype of a protocol (by using `# type: ignore` together with an explicit base class) when it is not strictly compatible, such as when it has an unsafe override.

Covariant subtyping of mutable attributes (#covariant-subtyping-of-mutable-attributes)

Rejected because covariant subtyping of mutable attributes is not safe. Consider this example:

```
class P(Protocol):
    x: float

def f(arg: P) -> None:
    arg.x = 0.42

class C:
    x: int

c = C()
f(c)  # Would typecheck if covariant subtyping
      # of mutable attributes were allowed.
c.x >> 1  # But this fails at runtime
```

It was initially proposed to allow this for practical reasons, but it was subsequently rejected, since this may mask some hard to spot bugs.

Overriding inferred variance of protocol classes (#overriding-inferred-variance-of-protocol-classes)

It was proposed to allow declaring protocols as invariant if they are actually covariant or contravariant (as it is possible for nominal classes, see PEP 484 (../pep-0484/)). However, it was decided not to do this because of several downsides:

- Declared protocol invariance breaks transitivity of sub-typing. Consider this situation:

```
T = TypeVar('T')

class P(Protocol[T]):  # Protocol is declared as invariant.
    def meth(self) -> T:
        ...
class C:
    def meth(self) -> float:
        ...
class D(C):
    def meth(self) -> int:
        ...
```

Now we have that `D` is a subtype of `C`, and `C` is a subtype of `P[float]`. But `D` is *not* a subtype of `P[float]` since `D` implements `P[int]`, and `P` is invariant. There is a possibility to "cure" this by looking for protocol implementations in MROs but this will be too complex in a general case, and this "cure" requires abandoning simple idea of purely structural subtyping for protocols.

- Subtyping checks will always require type inference for protocols. In the above example a user may complain: "Why did you infer `P[int]` for my `D`? It implements `P[float]`!". Normally, inference can be overruled by an explicit annotation, but here this will require explicit subclassing, defeating the purpose of using protocols.

- Allowing overriding variance will make impossible more detailed error messages in type checkers citing particular conflicts in member type signatures.

- Finally, explicit is better than implicit in this case. Requiring user to declare correct variance will simplify understanding the code and will avoid unexpected errors at the point of use.

Support adapters and adaptation (#support-adapters-and-adaptation)

Adaptation was proposed by PEP 246 (../pep-0246/) (rejected) and is supported by `zope.interface`, see the Zope documentation on adapter registries (https://web.archive.org/web/20160802080957/https://docs.zope.org/zope.interface/adapter.html). Adapters is quite an advanced concept, and PEP 484 (../pep-0484/) supports unions and generic aliases that can be used instead of adapters. This can be illustrated with an example of `Iterable` protocol, there is another way of supporting iteration by providing `__getitem__` and `__len__`. If a function supports both this way and the now standard `__iter__` method, then it could be annotated by a union type:

```
class OldIterable(Sized, Protocol[T]):
    def __getitem__(self, item: int) -> T: ...

CompatIterable = Union[Iterable[T], OldIterable[T]]

class A:
    def __iter__(self) -> Iterator[str]: ...
class B:
    def __len__(self) -> int: ...
    def __getitem__(self, item: int) -> str: ...

def iterate(it: CompatIterable[str]) -> None:
    ...

iterate(A())  # OK
iterate(B())  # OK
```

Since there is a reasonable alternative for such cases with existing tooling, it is therefore proposed not to include adaptation in this PEP.

Call structural base types “interfaces” (#call-structural-base-types-interfaces)

“Protocol” is a term already widely used in Python to describe duck typing contracts such as the iterator protocol (providing `__iter__` and `__next__`), and the descriptor protocol (providing `__get__`, `__set__`, and `__delete__`). In addition to this and other reasons given in specification (#specification), protocols are different from Java interfaces in several aspects: protocols don’t require explicit declaration of implementation (they are mainly oriented on duck-typing), protocols can have default implementations of members and store state.

Make protocols special objects at runtime rather than normal ABCs (#make-protocols-special-objects-at-runtime-rather-than-normal-abcs)

Making protocols non-ABCs will make the backwards compatibility problematic if possible at all. For example, `collections.abc.Iterable` is already an ABC, and lots of existing code use patterns like `isinstance(obj, collections.abc.Iterable)` and similar checks with other ABCs (also in a structural manner, i.e., via `__subclasshook__`). Disabling this behavior will cause breakages. If we keep this behavior for ABCs in `collections.abc` but will not provide a similar runtime behavior for protocols in `typing`, then a smooth transition to protocols will be not possible. In addition, having two parallel hierarchies may cause confusions.

Backwards Compatibility (#backwards-compatibility)

This PEP is fully backwards compatible.

Implementation (#implementation)

The `mypy` type checker fully supports protocols (modulo a few known bugs). This includes treating all the builtin protocols, such as `Iterable` structurally. The runtime implementation of protocols is available in `typing_extensions` module on PyPI.

References (#references)

[typing (#id1)]
https://docs.python.org/3/library/typing.html (https://docs.python.org/3/library/typing.html)
[wiki-structural (#id2)]
https://en.wikipedia.org/wiki/Structural_type_system (https://en.wikipedia.org/wiki/Structural_type_system)
[zope-interfaces (#id3)]
https://zopeinterface.readthedocs.io/en/latest/ (https://zopeinterface.readthedocs.io/en/latest/)
[abstract-classes (#id4)]
https://docs.python.org/3/library/abc.html (https://docs.python.org/3/library/abc.html)
[collections-abc (#id5)]
https://docs.python.org/3/library/collections.abc.html (https://docs.python.org/3/library/collections.abc.html)
[typescript (#id6)]
https://www.typescriptlang.org/docs/handbook/interfaces.html (https://www.typescriptlang.org/docs/handbook/interfaces.html)
[golang] *(1 (#id7), 2 (#id14))*
https://golang.org/doc/effective_go.html#interfaces_and_types (https://golang.org/doc/effective_go.html#interfaces_and_types)
[data-model (#id8)]

https://docs.python.org/3/reference/datamodel.html#special-method-names (https://docs.python.org/3/reference/datamodel.html#special-method-names)

[typeshed (#id10)]

https://github.com/python/typeshed/ (https://github.com/python/typeshed/)

[mypy (#id9)]

http://github.com/python/mypy/ (http://github.com/python/mypy/)

[several (#id11)]

https://mail.python.org/pipermail/python-ideas/2015-September/thread.html#35859 (https://mail.python.org/pipermail/python-ideas/2015-September/thread.html#35859)

[discussions (#id12)]

https://github.com/python/typing/issues/11 (https://github.com/python/typing/issues/11)

[elsewhere (#id13)]

https://github.com/python/peps/pull/224 (https://github.com/python/peps/pull/224)

Copyright (#copyright)

This document has been placed in the public domain.